



Diseño e implementación de un sistema RAG (Retrieval-Augmented Generation) en un LLM para la construcción de reglas YARA en la detección de malware

Autor

César Iván Quintero Garzón

Tutor

Ing. Jacinto Gómez Edwar MSC

Universidad Distrital Francisco José De Caldas

Maestría en Ciencias de la información y comunicaciones

Línea de Investigación en Teleinformática

Bogotá, Colombia

noviembre de 2025

Contenido

Contenido	2
RESUMEN.....	3
PALABRAS CLAVE	3
INTRODUCCIÓN.....	4
1. PROBLEMA DE INVESTIGACIÓN	7
1.1 PLANTEAMIENTO DEL PROBLEMA.....	7
1.2 FORMULACION DEL PROBLEMA.....	8
1.3 SISTEMATIZACIÓN DEL PROBLEMA.....	9
2. OBJETIVOS	9
2.1 OBJETIVO GENERAL.....	9
2.2 OBJETIVOS ESPECÍFICOS.....	10
3. JUSTIFICACIÓN.....	10
4. MARCO DE REFERENCIA	11
4.1 MARCO TEÓRICO	11
4.2 MARCO CONCEPTUAL.....	20
5. METODOLOGÍA.....	21
6. IMPACTO Y RESULTADOS ESPERADOS	23
7. RECURSOS Y PRESUPUESTO	24
8. CRONOGRAMA	25
9. LIMITACIONES	26
10. REFERENCIAS.....	27

RESUMEN

El proyecto tiene como propósito diseñar, implementar y evaluar un sistema RAG (Retrieval-Augmented Generation) en un contexto específico de ciberseguridad que combine dos elementos clave:

Un módulo de recuperación de información basado en una caracterización de muestras reales de malware, y un modelo de lenguaje (LLM) capaz de generar reglas YARA de forma automatizada y consistente.

El propósito es mejorar la detección de malware mediante el proceso de generación de reglas YARA. En la práctica, esto significa que el sistema no solo producirá firmas sintácticamente válidas, sino que también reflejará comportamientos específicos y la detección de cada familia de malware caracterizada.

PALABRAS CLAVE

YARA, Algoritmo Heurístico, Ciberseguridad, Detección de Malware, Ingeniería Reversa, Automatización, Firmas Digitales, Análisis Estático.

INTRODUCCIÓN

La detección de malware es uno de los mayores retos actuales en materia de ciberseguridad, debido a la rápida evolución de las amenazas y a las técnicas de evasión cada vez más sofisticadas que utilizan los ciberdelincuentes. Las reglas YARA se han consolidado como una herramienta esencial para identificar patrones maliciosos en archivos binarios, lo que permite a los analistas detectar amenazas mediante firmas específicas. Sin embargo, el proceso manual de creación de estas reglas es lento, depende de la experiencia del analista y es propenso a errores, lo que puede comprometer la eficacia de los sistemas de defensa. Esta situación crea una ventana de vulnerabilidad que puede ser explotada por actores maliciosos para desarrollar variantes de malware más resistentes y difíciles de detectar (Naik et al., 2019).

YARA se vincula convencionalmente con metodologías de detección basadas en firmas, que implican la identificación de patrones o firmas establecidos que se correlacionan con el malware. Este enfoque resulta ventajoso para la identificación de amenazas conocidas.

El personal de seguridad puede emplear las reglas YARA para buscar de forma proactiva malware dentro de un entorno mediante el examen de archivos, memoria y otros dominios en busca de posibles amenazas basadas en directrices establecidas. Cuando se incorpora a marcos de seguridad automatizados, YARA puede mejorar la eficiencia de los procesos de detección de malware, lo que reduce la carga manual de los analistas y acelera el tiempo de respuesta ante posibles amenazas. Las reglas YARA también pueden facilitar la categorización del malware en familias o clasificaciones distintas basadas en características comunes, lo que resulta fundamental para comprender el panorama de las amenazas y formular estrategias defensivas. A pesar de su eficacia, la formulación de reglas YARA eficaces plantea retos considerables. Requiere una comprensión profunda del comportamiento del malware y de las complejidades del código asociado (Si et al., 2022).

Además, a medida que el malware sigue evolucionando, las reglas existentes pueden quedar obsoletas o ser demasiado generales, lo que da lugar a casos de falsos positivos o falsos negativos. El proceso de búsqueda de reglas YARA, especialmente en conjuntos de datos extensos, puede requerir muchos recursos y, si no se administra con prudencia,

puede obstaculizar el rendimiento del sistema (Si et al., 2022). La creación precisa y el mantenimiento continuo de las reglas YARA exigen un nivel de experiencia necesario, lo que puede suponer una limitación para los analistas menos experimentados. En conclusión, las reglas YARA representan un elemento fundamental de las estrategias contemporáneas de detección de malware, ya que proporcionan una metodología versátil y robusta para la identificación de amenazas. Sin embargo, para optimizar su eficacia, es imprescindible un perfeccionamiento continuo y la participación de expertos para adaptarse a la naturaleza dinámica de las amenazas a la ciberseguridad.

Se han propuesto varios enfoques para automatizar la generación de reglas YARA, que son cruciales en un entorno de ciberseguridad, donde la velocidad y la eficiencia en la detección de amenazas son esenciales. Entre los métodos utilizados se encuentran los algoritmos heurísticos, que aplican principios de búsqueda eficientes para identificar patrones, y las técnicas de biclustering, que agrupan datos e identifican similitudes dentro de conjuntos de malware (Raff et al., 2020). También se ha explorado el uso de la lógica difusa, que puede manejar la incertidumbre y la imprecisión inherentes al comportamiento de ciertos tipos de malware, lo que permite una identificación que se adapta a ese comportamiento errático y da robustez a la solución gracias a la agrupación de características de comportamiento en los sistemas típicos de la lógica difusa (Naik, Jenkins, Savage, et al., 2020).

Además, se están integrando en este campo modelos lingüísticos avanzados, como BERT (Bidirectional Encoder Representations from Transformers), que proporcionan capacidades de procesamiento del lenguaje natural que permiten a las herramientas comprender mejor y generar texto en el contexto de la ciberseguridad (Xu & Qiao, 2022). Herramientas como YARGen y YaraGenerator (Naik, Jenkins, Cooke, et al., 2020), así como marcos innovadores como NeuroYara (Mansour et al., 2025) y APIARY (Coscia et al., 2025), han demostrado avances significativos en la generación automatizada de firmas, lo que permite a los analistas reducir la carga de trabajo manual. Sin embargo, estas herramientas siguen enfrentándose a retos en cuanto a escalabilidad, ya que deben adaptarse a volúmenes de datos cada vez mayores y a nuevas amenazas, con bajos niveles de precisión, dado que los falsos positivos o negativos pueden comprometer la correcta detección del sistema.

En este contexto, la técnica de generación aumentada por recuperación (RAG) ha surgido como una solución prometedora para superar algunas de estas limitaciones (Lewis et al., 2021). La RAG combina la recuperación de información contextual con la generación de lenguaje natural, lo que significa que no solo se basa en un modelo de lenguaje grande para generar respuestas, sino también en información relevante y actualizada para mejorar su calidad y relevancia. Esta sinergia permite a los modelos de lenguaje grandes (LLM) no solo producir firmas con mayor precisión, sino también adaptarse de manera sencilla a las nuevas variantes de malware, lo que proporciona un enfoque más sólido para la detección y respuesta a las amenazas de ciberseguridad (Zhang et al., 2025).

Este proyecto propone el diseño y la implementación de un sistema basado en la técnica de generación aumentada por recuperación (RAG) que se integra con un modelo de lenguaje grande (LLM) para automatizar la generación de reglas YARA, con el objetivo fundamental de mejorar la precisión, la cobertura y la eficiencia de la detección de malware en entornos informáticos (Jelodar et al., 2025). La relevancia de esta investigación radica en la creciente complejidad y diversidad de las amenazas cibernéticas actuales, que hacen imprescindible el desarrollo de herramientas capaces de adaptarse dinámicamente a las nuevas variantes de malware.

La solución propuesta se basa en la creación de un repositorio de muestras de malware, que se caracterizan mediante análisis estáticos (Raff et al., 2017). Este repositorio no solo servirá como base de datos de referencia, sino que también alimentará el módulo de recuperación del sistema RAG, lo que permitirá realizar consultas rápidas y relevantes. El análisis estático de las muestras proporciona información detallada sobre las características y el comportamiento de cada familia de malware, lo que enriquece el proceso de generación de reglas.

El sistema permitirá a los usuarios realizar consultas en lenguaje natural, transformando así la forma en que los analistas de seguridad interactúan con la tecnología. Basándose en estas consultas, el sistema recopilará la información relevante del repositorio para generar reglas YARA funcionales (Zhang et al., 2025). Estas reglas no serán fórmulas predefinidas, sino que se adaptarán a las características específicas de cada familia de malware, lo que garantizará una mayor eficacia en situaciones reales de detección.

Este enfoque integrado de RAG y LLM representa un avance significativo en la automatización de la generación de firmas de detección. Al permitir una interacción más natural y eficaz entre el usuario y el sistema, se espera que este proyecto no solo mejore la calidad de las reglas generadas, sino que también reduzca el tiempo y el esfuerzo que requieren los analistas para crear y actualizar las reglas YARA. En última instancia, se espera que el sistema aumente la capacidad de respuesta ante las amenazas cibernéticas, mejorando así la ciberseguridad en general.

El documento se organiza en cinco capítulos principales. En primer lugar, se presenta el planteamiento del problema y los objetivos del proyecto. Luego, se expone la justificación y el marco teórico, abordando los conceptos clave como YARA, LLM y RAG, así como los antecedentes relevantes. Posteriormente, se describe la metodología empleada, que incluye la recolección de muestras, análisis estático, diseño del sistema y evaluación de desempeño. Finalmente, se detallan los resultados esperados, el impacto del proyecto en el campo de la ciberseguridad, los recursos requeridos, el cronograma de trabajo y las limitaciones identificadas.

1. PROBLEMA DE INVESTIGACIÓN

1.1 PLANTEAMIENTO DEL PROBLEMA

Hoy en día, la ciberseguridad enfrenta serios peligros debido a los últimos adelantos en el malware y las tácticas innovadoras que usan para no ser detectados. YARA se ha vuelto fundamental para hallar patrones maliciosos mediante reglas específicas. Se ha observado que el proceso de hacer estas reglas de forma manual resulta lento, difícil y propenso a fallas, dependiendo de la experiencia de los analistas de ciberseguridad

causando cambios en la calidad de las estas. Este proceso manual crea un tiempo de indefensión que los cibercriminales pueden aprovechar para mejorar los malware. Por ende, es crucial crear métodos automatizados que agilicen la creación de reglas YARA, mejorando la exactitud y el impacto de las detecciones permitiendo identificar rápidamente las nuevas variantes de malwares junto con la respuesta a incidentes (Si et al., 2022).

Hoy en día ya hay desarrollos e implementaciones frente a la construcción de reglas YARA mediante la integración de aprendizaje automático, la integración de lógica difusa y el biclustering. Pero los desafíos como los volúmenes bajos de muestras de malware novedoso o más reciente, limitaciones frente a resultados con falsos positivos ocasionando menor efectividad en las detecciones y la degradación del desempeño ocasionan problemas en la detección temprana de malware sofisticado permitiendo su ejecución durante periodos prolongados. Estas situaciones pueden generar una sobrecarga e ineficiencia operativa en los equipos para la detección, análisis y respuesta de incidentes, también en la ventaja operativa para un ciberdelincuente generando avances en el desarrollo automatizado de malware, incremento de riesgos e impacto financiero de las victimas u organizaciones y la falta de confianza en la efectividad de la defensa (Naik, Jenkins, Cooke, et al., 2020).

Es importante generar soluciones que permitan generar estas reglas de forma automatizada con más agilidad, calidad, precisión y confiabilidad adaptándose a tiempos donde la inteligencia artificial ha tomado un gran protagonismo en la vida cotidiana, y también ha permitido que sea usada de forma delictiva para ocasionar daños en los ambientes cibernéticos por parte de ciberdelincuentes. Por eso es importante también usar estas herramientas para generar nuevos desarrollos para la protección de la información y la ciberseguridad.

1.2 FORMULACION DEL PROBLEMA

¿Cómo puede el diseño e implementación de un sistema de Retrieval-Augmented Generation (RAG) en un LLM mejorar la precisión, rendimiento y accesibilidad en la construcción de reglas YARA para la detección de malware, en comparación con otros métodos automatizados y el uso de LLMs sin aumentar?

1.3 SISTEMATIZACIÓN DEL PROBLEMA

- ¿Cuáles son las fuentes de datos más relevantes y cómo deben ser pre procesadas para construir una base de conocimiento efectiva para el dominio del malware en un LLM?
- ¿Cómo debe estructurarse el pipeline de RAG para recuperar de manera óptima los fragmentos de información más útiles a partir de una consulta en lenguaje natural de un analista?
- ¿Qué estrategia de prompting para el LLM es más efectiva para integrar el contexto recuperado y generar reglas YARA que sean funcionales para la identificación de malware?
- ¿Cuáles son las métricas (cuantitativas y cualitativas) en las cuales supera el sistema RAG propuesto con respecto a las reglas creadas de forma directa con un LLM sin RAG y a reglas creadas automatizadas con otros desarrollos?

2. OBJETIVOS

2.1 OBJETIVO GENERAL

Diseñar, implementar y evaluar un sistema RAG que combine un componente de recuperación basado en base de datos de malware y un LLM para generar reglas YARA automatizadas y de alta calidad para detección de malware.

2.2 OBJETIVOS ESPECÍFICOS

- Recolectar e Identificar muestras de diferentes familias de malware determinando el tipo de interés ransomware.
- Caracterizar las muestras de malware mediante un análisis estático con el fin de obtener atributos del fichero y permitir la construcción de un repositorio para la recuperación relevante.
- Diseñar e implementar un sistema RAG especificando los componentes de recuperación y generación.
- Diseñar prompts para la creación de reglas YARA que usen un contexto recuperado que permitan generarlas.
- Determinar las métricas de calidad, cobertura y efectividad para la detección de malware de las reglas YARA generadas en el RAG en comparación con las generadas en distintos LLM.
- Realizar las pruebas de las reglas YARA del RAG con las generadas en diferentes LLM y evaluar mediante las métricas definidas.

3. JUSTIFICACIÓN

En la detección y clasificación de malware, los analistas de ciberseguridad enfrentan un reto constante debido a que cada día aparecen miles de variantes nuevas, muchas diseñadas para burlar los mecanismos de defensa. Esta evolución continua de la capacidad de los ciberdelincuentes para generar mejores malware lleva a desarrollar sistemas más inteligentes y automatizados que ayuden a identificar amenazas de forma mucho más rápido antes de lograr sus objetivos.

Las reglas YARA son una herramienta clave en este proceso. Permiten describir característicos y/o únicos de los malware los cuales son esenciales para su detección. Crear estas reglas a mano exige experiencia, tiempo y precisión y donde un pequeño error puede dejar fuera una familia completa de amenazas. Además, la rapidez con que surgen nuevos códigos maliciosos creados por ciberdelincuentes supera la capacidad humana de generar y validar reglas actualizadas.

Aquí es donde entran en juego los modelos de lenguaje grandes (LLM) y las técnicas de Recuperación Aumentada por Generación (RAG). Esta metodología combina la capacidad de razonamiento de un LLM con un sistema de recuperación de información específico que le aporta datos reales y actuales según el contexto objetivo. Un sistema RAG construido para generar reglas YARA podría convertirse en una herramienta de mucha utilidad para los analistas de ciberseguridad. Además, se podría analizar como plataforma de investigación sobre cómo la inteligencia artificial puede integrarse de forma efectiva en la ciberseguridad permitiendo mejoras esperadas en precisión, cobertura y adaptabilidad frente a la detección de nuevas amenazas de malware.

Este proyecto busca entender mejor cómo se emplean los grandes modelos de lenguaje LLM en tareas de ciberseguridad y estrecha la relación entre investigación y trabajo diario. También busca mejorar la defensa contra ciberataques para proteger datos, infraestructuras clave y usuarios.

4. MARCO DE REFERENCIA

4.1 MARCO TEÓRICO

Las firmas Yara (**Y**et **A**nother **R**ecursive **A**cronym) consisten en un grupo de reglas las cuales permiten definir un patrón de código malicioso en cadenas de bytes, texto, expresiones regulares, operadores especiales, entre otros, que permiten definir la lógica

de un código permitiendo la identificación de malware. Se ha convertido en un estándar por la industria de la ciberseguridad para crear firmas de detección llegando a ser conocida como la navaja suiza de los investigadores de malware.

Yara fue desarrollado por Víctor Manuel Álvarez, también conocido como Plusvic, en el año 2013 y se mayor desarrollo ha sido el aporte en la plataforma virustotal.com.

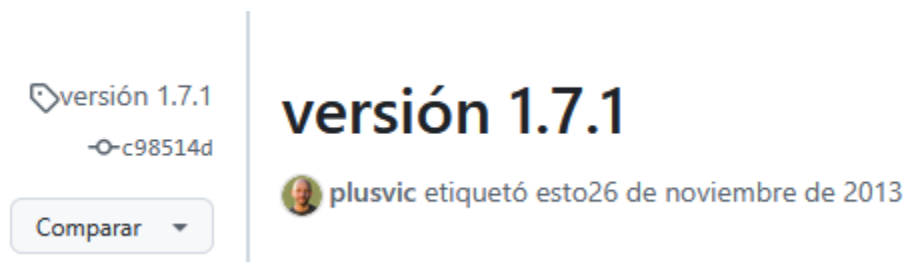


Figura 1. Primera versión de Yara en GitHub

(https://github.com.translate.googleusercontent.com/VirusTotal/yara/releases/tag/v1.7.1?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc)

Según (Naik et al., 2019) Las reglas YARA son una técnica de coincidencia de patrones desarrollada para el análisis de malware mediante ingeniería inversa, permitiendo la creación de reglas personalizadas basadas en descripciones de grupos de malware y cadenas o secuencias de bytes específicas. Son adaptables y eficaces para descubrir y clasificar malware.(Naik, Jenkins, Cooke, et al., 2020) también se menciona que estas reglas nacieron como una técnica flexible y personalizable para el análisis de malware, permitiendo a los analistas crear reglas basadas en requerimientos específicos del dominio de seguridad. Se escriben principalmente utilizando Indicadores de Compromiso (IOC) extraídos de malware a través de ingeniería inversa.

(S. Li et al., 2023) profundiza y analiza las reglas YARA como firmas escritas por personas capacitadas en temas de ciberseguridad utilizadas para identificar los empaquetados de malware. Estas reglas se basan en patrones binarios, pero su efectividad está limitada por los altos falsos positivos y la dependencia de la experiencia de los analistas, lo que requiere la generación automatizada para mejorar la precisión. Por eso es importante generar automáticamente reglas YARA robustas para la detección de

malware las cuales también se logre mejorar la precisión y reduzca los falsos positivos en la identificación de malware. Se debe centrar en las técnicas de ofuscación utilizadas al empaquetar el malware lo cual implica analizar las limitaciones de las reglas escritas por humanos existentes, particularmente sus altas tasas de falsos positivos. Por eso se reducir la dependencia de la experiencia de los analistas mediante el desarrollo de un sistema más robusto y automatizado para generar reglas de detección.

(Raff et al., 2020) presentaron un algoritmo automatizado que utiliza grandes n-gramas y un enfoque biclustering para generar eficientemente reglas YARA, reduciendo significativamente el tiempo que los especialistas dedican a la creación de reglas mientras mantiene tasas efectivas de verdaderos positivos y bajos falsos positivos. Este método permitió reducir el tiempo de construcción de reglas YARA en un 44-86% con bajas tasas de falsos positivos. Aun así, este algoritmo presenta limitaciones frente a la comprensión de su efectividad en diversos tipos de malware lo que plantea preocupaciones sobre la confiabilidad de las reglas generadas en aplicaciones prácticas, además que no se demuestra su integración con herramientas de ciberseguridad existentes y su práctica en la realidad.

(Si et al., 2022) proponen un método para la generación automatizada de reglas YARA extrayendo secuencias de llamados del API y parámetros de tiempo de ejecución, utilizando algoritmos aleatorios de regresión logística para calcular los pesos de cada característica, mejorando la eficacia y automatización en la detección de malware. Este método permite generar reglas YARA a partir de características de comportamiento dinámico de muestras, pero no aborda la escalabilidad del método cuando se aplica a conjuntos de datos más grandes de malware y muestras benignas, lo cual es crucial para las aplicaciones del mundo real. Falta explorar la adaptabilidad del método propuesto a nuevas variantes de malware.

(Naik, Jenkins, Cooke, et al., 2020) realizaron una evaluación de tres herramientas automatizadas para generar reglas YARA: YARGen, YaraGenerator y yabin. Cada herramienta emplea metodologías diferentes, pero todas requieren un procesamiento posterior para mejorar la eficacia, ya que las reglas generadas automáticamente pueden no ser tan efectivas como las elaboradas manualmente. Se realizaron mejoras a las tres

herramientas mediante la aplicación de hash difuso encontrando que la herramienta YarGen superó a YaraGenerator y yabin en los resultados de detección.

(Xu & Qiao, 2022) analizan un modelo de lenguaje de cuerdas basado en Bert (beslm) que automatiza la generación de reglas YARA, mejorando su calidad mediante el aprendizaje efectivo de la información semántica y las asociaciones de cadenas, lo que resulta en métricas de detección mejoradas en comparación con los métodos tradicionales. Para esto se requiere una fase de pre-entrenamiento para aprender información semántica de las reglas YARA y las asociaciones de cuerdas. Utilizan una fase de afinación para filtrar cadenas de alta calidad para mejorar la eficacia de la detección. Como metodología se centran en mejorar las métricas de detección mediante el aprovechamiento de las capacidades del modelo beslm. Esto permitió una mejora de 13.3%, 13.8% y 15.1% en promedio. La investigación no investiga la escalabilidad del modelo propuesto cuando se aplica a conjuntos de datos más grandes o escenarios de malware más complejos, no se plantea cómo se puede adaptar para sistemas de detección en tiempo real y no se menciona el impacto potencial de la evolución de las técnicas de código malicioso en la eficacia de las reglas YARA generadas por el modelo beslm a lo largo del tiempo.

(Mr. Shreyas Biju Nair et al., 2024) presentan un framework en la web que genera automáticamente reglas YARA para archivos maliciosos cargados por los usuarios. Este enfoque mejora la portabilidad y la eficiencia en la detección de malware, logrando una precisión promedio de 0.80 en la identificación de muestras maliciosas. En la investigación existe una falta de discusión sobre la adaptabilidad de las reglas YARA generadas a las amenazas de malware en evolución, lo cual es crucial para mantener la relevancia en la detección de malware, la precisión de 0.80 mencionada en los resultados de las pruebas plantea interrogantes sobre el potencial de falsos positivos y falsos negativos, que siguen sin resolverse. No se explora el desempeño del marco en escenarios del mundo real, más allá de las pruebas de prototipos, lo que no deja claro su aplicación práctica.

(S. Li et al., 2023) analiza un enfoque automatizado para generar reglas YARA robustas destinadas a detectar con precisión el malware empaquetado titulado PackGenome. Este método reduce la dependencia de especialistas humanos, minimizando los esfuerzos

manuales y abordando las altas tasas de falsos positivos de las reglas tradicionales escritas por humanos. Para esto se centra en las técnicas de ofuscación utilizadas en el empaquetado binario, haciendo énfasis en la dependencia de patrones binarios específicos para identificar empaquetadores, que son cruciales para la disección de malware. El enfoque actual en la búsqueda de patrones de bytes en lugar de instrucciones sugiere una falta de metodologías que puedan analizar e interpretar de manera efectiva el comportamiento subyacente del malware empaquetado.

(Patil et al., 2025) se centraron en la elaboración de reglas YARA efectivas para la detección de malware, haciendo énfasis en la coincidencia precisa de patrones y la conciencia del contexto, además se destaca la importancia de la optimización de reglas y la adaptación a las amenazas de malware en evolución. Exploran la escritura efectiva de reglas YARA para la detección de malware y evalúan estrategias para mejorar la precisión y la eficiencia mediante el reconocimiento de patrones y estrategias de optimización de reglas y el manejo de técnicas evasivas empleadas por el malware. Este estudio dio como resultado que la combinación de estrategias mejora significativamente las tasas de detección y las actualizaciones periódicas minimizan los falsos positivos en la detección de malware.

(Naik, Jenkins, Savage, et al., 2020) incorporan la aplicación de reglas difusas con reglas YARA para mejorar el rendimiento durante el análisis de malware, haciendo énfasis en las limitaciones de las expresiones booleanas en condiciones complejas en lugar de enfoques heurísticos logrando una mejora de la tasa de detección de similitud en 11.3% con respecto a las reglas estándar de YARA.

(Naik et al., 2021) analizaron las reglas YARA mejoradas y las reglas YARA integradas, que utilizan hash difuso y reglas difusas para mejorar los resultados de detección de malware. Estos métodos mejoran la efectividad de las reglas YARA existentes sin aumentar significativamente la complejidad o los gastos generales asociados con la creación de reglas encontrando que las reglas YARA mejoradas mejoraron ligeramente la tasa de detección a 67.1 y las reglas básicas de YARA se desempeñaron ligeramente mejor que los métodos de hash difuso.

(Bilstein & Plohmann, 2019) desarrollan YARA-signator, un marco diseñado para la generación automatizada de reglas YARA basadas en código, abordando el desafío de la creación manual de firmas en el análisis de malware. El enfoque busca identificar fragmentos de código característicos que son exclusivos de una familia específica de malware, que aparecen frecuentemente dentro de esa familia, pero no en otras. Se basa en aislar n-gramos—secuencias de instrucciones de tamaño fijo. YARA-Signator emplea un flujo de trabajo estructurado de varias etapas que procesa sistemáticamente los datos para lograr sus objetivos.

(Si et al., 2022) analizaron la generación automatizada de reglas YARA basado en comportamientos dinámicos, específicamente utilizando secuencias de llamadas API y parámetros de tiempo de ejecución para crear reglas heurísticas para la detección efectiva de malware, mejorando la automatización y la precisión en la identificación de muestras maliciosas. Esto se hizo mediante la utilización de bosques aleatorios y algoritmos de regresión logística en YaraML.

(Siddabathula et al., 2023) analizan las reglas YARA como una herramienta para la detección robusta basada en reglas de ataques a la red. Las reglas heurísticas de YARA se utilizan para analizar el tráfico de la red, identificando paquetes maliciosos mediante la aplicación de patrones predefinidos, mejorando así el proceso de detección y alerta para los administradores de red.

(Bharti et al., 2023) usaron reglas heurísticas de YARA para la detección de malware mediante la definición de patrones para identificar software malicioso. El documento analiza su estructura sintáctica, propiedades e implementación en Android, lo que ayuda en la detección eficiente de aplicaciones maliciosas a través de la creación de reglas específicas. Para esto usaron Androguard y Cuckoo para la creación de reglas.

(S. Li et al., 2023) ejecutan un enfoque especializado en detectar packers de malware mediante firmas heurísticas, esto mediante la identificación de 'genes' de packers para realizar un ensamblaje heurístico de las reglas YARA. Este enfoque es muy especializado y no cubre otros tipos de malware más que el probado.

(H. Li & Li, 2024) realizan la generación de reglas YARA a partir de indicadores de riesgo con métricas de calidad. El objetivo es traducir información de riesgo en reglas automáticas confiables mediante la extracción de indicadores, priorización heurística y evaluación mediante métricas de razonabilidad y eficacia. Este estudio necesita pruebas en entornos reales, las métricas propuestas aún tienen resultados preliminares y tiene riesgo de sobreajuste a datasets de laboratorio.

(Mansour et al., 2025) El objetivo principal de este estudio es desarrollar un generador de reglas YARA eficiente, automático y de alta calidad capaz de igualar el rendimiento de analistas expertos en malware, manteniendo al mismo tiempo una baja tasa de falsos positivos. Para ello utiliza dos redes neuronales de aprendizaje a clasificar para comprender la efectividad y las correlaciones entre n-gramas para la construcción de reglas. Esto incluye un modelo discriminativo bidireccional de n-gramas de memoria larga a corto plazo (LSTM) para aprender los significados semánticos y maliciosidad de las secuencias de bytes sin intervención humana. El sistema captura automáticamente la maliciosidad de los n-gramas a través de un novedoso modelo discriminativo de aprendizaje a rango, incorporando información de contexto y relaciones entre n-gramas. NeuroYara supera las herramientas existentes y las reglas generadas manualmente en términos de predicción de malware y pruebas de falsos positivos en más de 90 familias de malware. Logra esto mientras genera reglas con una baja tasa de falsos positivos. Existe una amenaza potencial de que el modelo de n-gramas humanos de NeuroYara pueda aprender características léxicas de reglas YARA de acceso público definidas por humanos en lugar de los patrones inherentes de binarios maliciosos y benignos.

(Coscia et al., 2025) presentan APIARY, un innovador generador de reglas automático basado en API diseñado para mejorar la identificación de malware a través de firmas personalizadas. Se enfoca en diferenciar el malware del software legítimo analizando patrones específicos en el uso de las API. La herramienta es efectiva independientemente de si los datos de entrada provienen de análisis dinámicos o estáticos de archivos ejecutables similares en Windows. APIARY clasifica las muestras de malware en modelos “regulares”, “regulares leves” y “no regulares” según sus características de API, se caracterizan los patrones del malware con precisión mediante la identificación y

eliminación de modelos redundantes. Al final el algoritmo convierte los modelos API en reglas YARA.

(Zhang et al., 2025) presentan RULELLM, una novedosa herramienta diseñada para generar automáticamente reglas YARA y Semgrep para detectar paquetes de software malicioso en ecosistemas de Software de Código Abierto (OSS). Tiene como objetivo aprovechar los modelos de lenguaje de gran tamaño (LLM) para automatizar la generación de reglas, mejorando así las capacidades de detección y análisis de seguridad para amenazas conocidas y emergentes. RULELLM extrae metadatos (dependencias, URL, versiones, nombres de paquetes, descripciones) de paquetes maliciosos, tratándolos como entrada LLM. Estos metadatos a menudo difieren entre paquetes maliciosos y benignos, y los maliciosos tienen potencialmente información de autor mínima/falsa, dependencias excesivas/inusuales o nombres tipografiados. El sistema generó 763 reglas en total (452 reglas YARA y 311 reglas Semgrep), las cuales están bien estructuradas y se pueden implementar directamente en las herramientas de seguridad existentes. Las reglas RULELLM detectaron con éxito variantes de malware OSS, con una tasa de detección general de 90.32% y una tasa de detección promedio de 96.62%. El desempeño de RULELLM depende en gran medida de la capacidad de razonamiento del LLM, además tiene dificultades para manejar código malicioso excesivamente largo, como código ofuscado o cargas útiles codificadas en base64.

(Lewis et al., 2021) presentan el modelo de Retrieval-Augmented Generation (RAG), que combina memoria paramétrica y no paramétrica para la generación del lenguaje, con el objetivo de afrontar las limitaciones de los modelos lingüísticos pre entrenados habituales en tareas intensivas en conocimiento, además de expandir o revisar fácilmente la memoria y la falta de conocimiento de las predicciones. El modelo RAG combinan un modelo seq2seq pre entrenado (memoria paramétrica) con un índice vectorial denso (no paramétrica) al que accede un recuperador neuronal pre entrenado. La arquitectura se basa en un recuperador y un generador que determina estos documentos y la entrada para producir salida. Como conclusión del estudio determinan que la memoria no paramétrica se puede actualizar fácilmente en el momento de la prueba reemplazando el índice, haciendo que RAG se adapte a los cambios sin volver a entrenar.

(Pal Singh Shobhit, 2024) exploran la integración de Retrieval-Augmented Generation (RAG) con herramientas de inteligencia de amenazas en ciberseguridad para automatizar el análisis e interpretación de las amenazas. El objetivo principal es analizar cómo se puede aprovechar RAG para analizar grandes cantidades de datos de diversas fuentes, incluidos registros, fuentes de amenazas e informes de incidentes, para generar información procesable. El estudio tiene como objetivo instruir cómo la automatización del análisis de inteligencia de amenazas con RAG representa un paso fundamental en la lucha contra las amenazas cibernéticas, brindando un mecanismo de defensa más resiliente y proactivo para las organizaciones. La implementación de RAG aumenta sustancialmente la precisión del análisis de inteligencia de amenazas al procesar grandes cantidades de datos y generar información contextual. Reduce los falsos positivos y acelera drásticamente la recopilación y el análisis de datos, lo que permite respuestas más ágiles a las amenazas. Si bien RAG puede mermar los falsos positivos, existe el riesgo de imprecisiones debido a sesgos en los datos de entrenamiento, además la integración con herramientas de ciberseguridad existentes puede ser difícil debido a que se requiere modificaciones técnicas.

(Al-Karaki et al., 2024) en este estudio se propone discutir cómo se pueden utilizar los modelos de lenguaje grande (LLM) para detectar patrones camuflados dentro del código del malware que podrían ser difíciles de identificar para los analistas de ciberseguridad. Para ello proponen entrenar LLM específicos en malware con el fin de detectar rasgos lingüísticos, es necesario un re entrenamiento continuo de los modelos de detección de amenazas, realizar un monitoreo del rendimiento de los LLM, así como el ajuste y actualización de los datos de entrenamiento y la integración con herramientas existentes como antivirus o IDS. Se muestra la preocupación de que el desarrollo de LLM entrenado específicamente con malware se use con fines inseguros y también se menciona que la información y literatura existente aborda casos con los LLM ChatGPT o Gemini lo cual muestra una oportunidad de trabajar con otros LLM en esta área de la ciberseguridad.

4.2 MARCO CONCEPTUAL

- **Detección de Malware:** Proceso de identificación de software malicioso en un sistema informático. La agilidad, precisión y fiabilidad es importante para la ciberseguridad para contrarrestar nuevas amenazas.
- **Reglas YARA:** Son un conjunto de patrones que permite identificar y clasificar familias de malware basándose en cadenas de texto, expresiones regulares y condiciones lógicas en archivos fuentes, binarios o en memoria permitiendo la detección de malware.
- **Modelos de Lenguaje Grande (LLM):** Son modelos de inteligencia artificial o aprendizaje profundo que son entrenados con grandes cantidades de datos lo cual les permite analizar secuencias de texto y extraer su significado. Los LLM permiten mediante la inteligencia artificial generativa producir contenido basado en instrucciones en lenguaje humano.
- **Retrieval-Augmented Generation (RAG):** Es una técnica o sistema que permite conectar un conjunto de datos externos con un modelo de lenguaje grade (LLM) con el fin de producir contenido, respuestas especializadas y contextualizadas a dominios específicos.
- **Malware:** Es un tipo de software malicioso cuyo objetivo es afectar sistemas informáticos con el fin de obtener información sensible no autorizada. Como ejemplo de malware están los gusanos, troyanos, spyware y ransomware.

5. METODOLOGÍA

La metodología contempla 5 fases principales:

Fase 1: Recolección e identificación de muestras de malware

En esta fase nos enfocaremos en reunir la mayor cantidad de muestras de diferentes familias de malware de ransomware. Para ello se hará una revisión de repositorios públicos, privados y académicos de muestras de malware.

Fase 2: Caracterización de las muestras mediante un análisis estático

En esta fase se construirá un repositorio con la información estructural y semántica de las muestras que permitan identificar patrones y sirvan para el sistema de recuperación del RAG. Este análisis estático permitirá extraer atributos como hashes, metadatos, strings legibles, formato del archivo, bibliotecas usadas y firmas conocidas.

Fase 3: Diseño e implementación del sistema RAG

Se desarrollará la arquitectura que conecte la base de conocimiento (repositorio de caracterización de atributos de las muestras de los malware) con el modelo de lenguaje grande (LLM) para la generación contextual de reglas YARA. Para esto se debe definir los módulos de recuperación y generación, así como las herramientas a emplearse para esto.

Fase 4: Diseño de prompts para la creación de reglas YARA

En esta fase se diseñarán y pondrán a prueba varios tipos de prompts para guiar al modelo de lenguaje en la creación de reglas YARA que sean precisas, coherentes y correctas desde el punto de vista sintáctico. Por ejemplo, se evaluarán prompts que indiquen explícitamente el tipo de malware o los patrones de código que deben reflejarse en la regla.

Fase 5: Evaluación y métricas de desempeño

Se evaluará cuantitativamente el desempeño del sistema RAG frente a diferentes LLM, midiendo la efectividad de las reglas YARA generadas para la detección de malware comparando con un parámetro de referencia.

6. IMPACTO Y RESULTADOS ESPERADOS

Contribuir al campo de la ciberseguridad proponiendo avances en la generación de reglas YARA de forma automatizada para la identificación de malware, esto mediante la aplicabilidad de un RAG en un dominio específico de la ciberseguridad.

Se espera obtener unas métricas de desempeño TPR (True Positive Rate): $\geq 80\%$ en conjuntos de prueba mixtos y FPR (False Positive Rate): $\leq 5\%$ en software benigno. También realizar una comparación entre las reglas YARA construidas con el prototipo y las reglas manuales, demostrando su eficacia.

Resultados esperados:

- Sistema RAG funcional integrado con repositorio de caracterización de malware mediante análisis estático que genere reglas YARA automatizadas.
- Evidencia experimental de mejora en precisión y tiempo frente a reglas generadas en un LLM.
- Reglas YARA generadas en el LLM y verificadas.

7. RECURSOS Y PRESUPUESTO

A continuación, se relaciona los recursos y presupuesto para realizar este proyecto. La fuente de estos será personal.

Tipo de recurso	Recurso	Costo
Humanos	Director del proyecto	\$ 4.000.000
	Investigador	\$ 2.500.000
Tecnológicos	Computador de alto rendimiento (CPU/GPU)	\$ 5.000.000
Materiales	Material de oficina e impresión de informes	\$ 300.000
Operativos	Conectividad / internet	\$ 400.000
Total		\$ 12.200.000

8. CRONOGRAMA

Fase / Actividad principal	Mes 1	Mes 2	Mes 3	Mes 4	Mes 5
1. Recolección e identificación de muestras de malware					
2. Caracterización y análisis estático					
3. Diseño e implementación del sistema RAG					
4. Diseño de <i>prompts</i> y generación de reglas YARA					
5. Evaluación y validación del sistema					
6. Documentación, ajustes y entrega final					

9. LIMITACIONES

El acceso a bases de datos actualizadas y muestras reales de malware debido a restricciones por políticas de seguridad podría limitar la cantidad de familias que se podrían analizar en el RAG afectando la efectividad de las reglas YARA generadas. Para esto se debe realizar una búsqueda de repositorios públicos y confiables que permitan garantizar una cantidad suficiente de muestras para el desarrollo del proyecto.

10. REFERENCIAS

- Al-Karaki, J., Khan, M. A.-Z., & Omar, M. (2024). *Exploring LLMs for Malware Detection: Review, Framework Design, and Countermeasure Approaches* (No. arXiv:2409.07587). arXiv. <https://doi.org/10.48550/arXiv.2409.07587>
- Bharti, P., Roy, S. S., & Suresh, A. (2023). Implementation of Yara Rules in Android. *2023 International Conference on Computer Communication and Informatics (ICCCI)*, 1-5. <https://doi.org/10.1109/ICCCI56745.2023.10128288>
- Bilstein, F., & Plohmann, D. (2019). YARA-Signator: Automated Generation of Code-based YARA Rules. *The Journal on Cybercrime and Digital Investigations*, 5(1), 1-13. <https://doi.org/10.18464/CYBIN.V5I1.24>
- Coscia, A., Lorusso, R., Maci, A., & Urbano, G. (2025). APIARY: An API-based automatic rule generator for yara to enhance malware detection. *Computers & Security*, 153, 104397. <https://doi.org/10.1016/j.cose.2025.104397>
- Jelodar, H., Bai, S., Hamed, P., Mohammadian, H., Razavi-Far, R., & Ghorbani, A. (2025). *Large Language Model (LLM) for Software Security: Code Analysis, Malware Analysis, Reverse Engineering* (No. arXiv:2504.07137). arXiv. <https://doi.org/10.48550/arXiv.2504.07137>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2021). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* (No. arXiv:2005.11401). arXiv. <https://doi.org/10.48550/arXiv.2005.11401>
- Li, H., & Li, J. (2024). Automatically Generate Malware Detection Rules By Extracting Risk Information. *Proceedings of the 2024 5th International Conference on Computing,*

- Networks and Internet of Things*, 595-599.
<https://doi.org/10.1145/3670105.3670209>
- Li, S., Ming, J., Qiu, P., Chen, Q., Liu, L., Bao, H., Wang, Q., & Jia, C. (2023). PackGenome: Automatically Generating Robust YARA Rules for Accurate Malware Packer Detection. *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 3078-3092.
<https://doi.org/10.1145/3576915.3616625>
- Mansour, Z., Ou, W., Ding, S. H. H., Zulkernine, M., & Charland, P. (2025). NeuroYara: Learning to Rank for Yara Rules Generation Through Deep Language Modeling and Discriminative N-Gram Encoding. *IEEE Transactions on Dependable and Secure Computing*, 22(2), 1747-1762. <https://doi.org/10.1109/TDSC.2024.3449641>
- Mr. Shreyas Biju Nair, Mr. Laalas Tadavarthy, Mr. Kailas M K, & Mr. Gowrishankar T O. (2024). YARWEB: WEB-BASED GENERIC YARA RULE GENERATOR. *EPRA International Journal of Research & Development (IJRD)*, 295-302.
<https://doi.org/10.36713/epra15953>
- Naik, N., Jenkins, P., Cooke, R., Gillett, J., & Jin, Y. (2020). Evaluating Automatically Generated YARA Rules and Enhancing Their Effectiveness. *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, 1146-1153.
<https://doi.org/10.1109/SSCI47803.2020.9308179>
- Naik, N., Jenkins, P., Savage, N., Yang, L., Naik, K., & Song, J. (2019). Augmented YARA Rules Fused With Fuzzy Hashing in Ransomware Triaging. *2019 IEEE Symposium Series on Computational Intelligence (SSCI)*, 625-632.
<https://doi.org/10.1109/SSCI44817.2019.9002773>
- Naik, N., Jenkins, P., Savage, N., Yang, L., Naik, K., & Song, J. (2020). Embedding Fuzzy Rules with YARA Rules for Performance Optimisation of Malware Analysis. *2020*

- IEEE International Conference on Fuzzy Systems (FUZZ-IEEE)*, 1-7.
<https://doi.org/10.1109/FUZZ48607.2020.9177856>
- Pal Singh Shobhit, J. (2024). Automating Threat Intelligence Analysis with Retrieval Augmented Generation (RAG) for Enhanced Cybersecurity Posture. *International Journal of Science and Research (IJSR)*, 13(5), 251-255.
<https://doi.org/10.21275/SR24502103758>
- Patil, V., M, N. K., M, P. S., & Singh, A. (2025). Effectively Writing YARA Rules to Detect Malware. *International Journal for Research in Applied Science and Engineering Technology*, 13(1), 1265-1273. <https://doi.org/10.22214/ijraset.2025.66535>
- Raff, E., Sylvester, J., & Nicholas, C. (2017). *Learning the PE Header, Malware Detection with Minimal Domain Knowledge*. <https://doi.org/10.48550/ARXIV.1709.01471>
- Raff, E., Zak, R., Lopez Munoz, G., Fleming, W., Anderson, H. S., Filar, B., Nicholas, C., & Holt, J. (2020). Automatic Yara Rule Generation Using Biclustering. *Proceedings of the 13th ACM Workshop on Artificial Intelligence and Security*, 71-82.
<https://doi.org/10.1145/3411508.3421372>
- Si, Q., Xu, H., Tong, Y., Zhou, Y., Liang, J., Cui, L., & Hao, Z. (2022). Malware Detection Using Automated Generation of Yara Rules on Dynamic Features. En C. Su, K. Sakurai, & F. Liu (Eds.), *Science of Cyber Security* (Vol. 13580, pp. 315-330). Springer International Publishing. https://doi.org/10.1007/978-3-031-17551-0_21
- Siddabathula, K. S., Panneerselvam, R. K., Vasana, V., Vejendla, J., Rafi, M., & Gummadi, S. B. (2023). YaraCapper – YARA Rule-Based Automated System to Detect and Alert Network Attacks. En A. Verma, P. Verma, K. K. Pattanaik, & L. Garg, *Research Advances in Network Technologies* (1.^a ed., pp. 25-47). CRC Press. <https://doi.org/10.1201/9781003320333-3>

Xu, L., & Qiao, M. (2022). Yara rule enhancement using Bert-based strings language model. *2022 5th International Conference on Advanced Electronic Materials, Computers and Software Engineering (AEMCSE)*, 221-224.
<https://doi.org/10.1109/AEMCSE55572.2022.00052>

Zhang, X., Chen, H., He, Y., Niu, W., & Li, Q. (2025). *Automatically Generating Rules of Malicious Software Packages via Large Language Model*.
<https://doi.org/10.48550/ARXIV.2504.17198>